1. FCFS Scheduling algorithm

```c
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

2. SJF_NONpreemptive

```c
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort burst times
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

3. Priority

```c
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
printf("Enter burst time and priority:\n");

for (i = 0; i < n; i++) {
   printf("P%d Burst Time: ", i + 1);
   scanf("%d", &bt[i]);

   printf("P%d Priority: ", i + 1);
   scanf("%d", &priority[i]);
}

// Sort according to priority
for (i = 0; i < n - 1; i++) {
   for (j = i + 1; j < n; j++) {

      if (priority[i] > priority[j]) {

         temp = priority[i];
         priority[i] = priority[j];
         priority[j] = temp;

         temp = bt[i];
         bt[i] = bt[j];
         bt[j] = temp;
      }
   }
}

// Waiting time
wt[0] = 0;

for (i = 1; i < n; i++) {
   wt[i] = wt[i - 1] + bt[i - 1];
}

// Turnaround time
for (i = 0; i < n; i++) {
   tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

for (i = 0; i < n; i++) {
   printf("P%d\t%d\t%d\t\t%d\t%d\n",
         i + 1, bt[i], priority[i], wt[i], tat[i]);
}

return 0;
```

}

4. ROUND ROBIN

```
#include <stdio.h>

int main() {
   int n, i;
   int bt[20], rem_bt[20];
   int wt[20], tat[20];
   int quantum;
   int time = 0;
   int done;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter burst time:\n");

   for (i = 0; i < n; i++) {
      printf("P%d: ", i + 1);
      scanf("%d", &bt[i]);

      rem_bt[i] = bt[i];
      wt[i] = 0;
   }

   printf("Enter time quantum: ");
   scanf("%d", &quantum);

   // Round Robin
   while (1) {

      done = 1;

      for (i = 0; i < n; i++) {

         if (rem_bt[i] > 0) {

            done = 0;

            if (rem_bt[i] > quantum) {
               time = time + quantum;
               rem_bt[i] = rem_bt[i] - quantum;
            }
            else {
               time = time + rem_bt[i];
```

```
                wt[i] = time - bt[i];

                rem_bt[i] = 0;
            }
        }
    }

    if (done == 1)
        break;
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
           i + 1, bt[i], wt[i], tat[i]);
}

return 0;
}
```

Week 5 — Scheduling Execution Example

Illustrative terminal output with the requested username (generated example).

